/gpu-train Command Reference

> All commands are invoked via `/gpu-train` in Claude Code. Platform auto-detected from working directory.

Platform Overview

Server	SSH	GPU	Use Case	User
`spark`	`spark` (alias)	NVIDIA GB10 (ARM, 92GB)	Booster K1, Unitree G1/H1	`zentek`
`rtx`	`phh@192.168.120.155` (VPN)	8x RTX 6000D (x86, 85GB each)	MagicBot Z1 Locomotion	`phh`

Auto-detect rules:

- Working directory contains `Magicbot_Z1` or `magiclab_rl_lab` → `rtx`
- Working directory contains `booster_train` or `Spark` → `spark`

VPN:

- RTX: iNode VPN (`C:\Program Files (x86)\iNode\iNode Client\iNode Client.exe`) — auto-launch + retry every 3s × 10
- Spark: aTrust VPN (`C:\Users\Public\Desktop\aTrust.lnk`) — auto-launch + retry every 5s × 6

Command Quick Reference

Command	Purpose	Server
(no args)	`--status` + `--tail` combined	Both
`--status`	Check if training process is alive	Both
`--tail`	Show last 30 lines of training log	Both
`--live`	Real-time training output	Both
`--gpu`	Full GPU utilization (nvidia-smi)	Both

Command	Purpose	Server
`--idle`	Find idle GPUs for new training	Both
`--mycuda`	Show my CUDA processes only	Both
`--models`	List saved model checkpoints	Both
`--check`	Training health trend analysis	Both
`--monitor`	Overfitting detection & best model finder	RTX
`--sim`	Record simulation video (Isaac Sim + MuJoCo)	RTX
`--start`	Launch / resume training	Both
`--play`	Live play on Spark display	Spark
`--kill`	Stop training process	Both
`--automation`	5-phase automated pipeline management	RTX
`--connect`	Test server connectivity + VPN auto-launch	Both
`--update`	Update training log markdown file	Both
`--compare`	Video comparison grid (deprecated, use `/merge`)	—

Detailed Command Reference

1. `--status` — Check Training Process

What: Check if training processes are alive on the remote server.

Execution:

```
# RTX
ssh phh@192.168.120.155 'ps aux | grep train.py | grep -v grep'

# Spark
ssh spark 'ps aux | grep -E "train_g1_pickplace|train.py" | grep -v grep'
```

Output: Process info (PID, command, elapsed time) or empty if not running.

2. `--tail` — Recent Training Logs

What: Show the last 30 lines of the active training log. Auto-detects which training is running first (`--status`), then fetches the corresponding log.

Log sources:

- **Spark**: tmux capture (`tmux capture-pane -t g1_train -p -S -50 | tail -30`) or log file
- **RTX**: `/tmp/z1_train_v1.log` or detect from process args

3. `--live` — Real-time Training Output

What: Show the last 30 lines of the tmux session (Spark) or tail the nohup log (RTX). Streams output.

4. `--gpu` — GPU Usage

What: Full GPU utilization including per-GPU memory and running processes.

Execution:

```
ssh <SSH_HOST> 'nvidia-smi'
```

5. `--idle` — Find Idle GPUs

What: Find GPUs suitable for launching a new training run.

Execution:

```
ssh <SSH_HOST> 'nvidia-smi --query-gpu=index,name,utilization.gpu,memory.used,memory.total --format=csv
```

Idle criteria (both must be true):

- GPU utilization < 10%
- Memory used < 5% of total (or < 5 GB)

Output example:

```
=== GPU Idle Check (RTX Server) ===

  GPU  Status     Util%  Mem Used / Total
  0    IDLE       0%     0.1G / 85.7G
  1    IDLE       0%     0.1G / 85.7G
  2    busy       1%     64.3G / 85.7G
  ...

Idle GPUs: 0, 1  (85.6 GB free each)
Suggested: --device cuda:0
```

6. `--mycuda` — Show My CUDA Usage

What: Show only CUDA processes belonging to the current user, excluding all other users.

RTX (user: `phh`):

```
ssh phh@192.168.120.155 '
map=$(nvidia-smi --query-gpu=index,gpu_bus_id --format=csv,noheader)
echo "=== My CUDA GPUs (user: phh) ==="
echo ""
nvidia-smi --query-compute-apps=gpu_bus_id,pid,process_name,used_memory --format=csv,noheader 2>/dev/nu
    bus=$(echo $bus | tr -d " ")
    pid=$(echo $pid | tr -d " ")
    if ps -o user= -p $pid 2>/dev/null | grep -q "phh"; then
        idx=$(echo "$map" | grep "$bus" | cut -d, -f1 | tr -d " ")
        name=$(echo $name | tr -d " ")
        echo "  cuda:$idx  PID=$pid  $name  $mem"
    fi
done
if [ -z "$(nvidia-smi --query-compute-apps=gpu_bus_id --format=csv,noheader 2>/dev/null)" ]; then
    echo "  (no compute processes on any GPU)"
fi
'
```

Spark (user: `zentek`):

```
ssh spark '
map=$(nvidia-smi --query-gpu=index,gpu_bus_id --format=csv,noheader)
echo "=== My CUDA GPUs (user: zentek) ==="
echo ""
nvidia-smi --query-compute-apps=gpu_bus_id,pid,process_name,used_memory --format=csv,noheader 2>/dev/nu
    bus=$(echo $bus | tr -d " ")
    pid=$(echo $pid | tr -d " ")
    if ps -o user= -p $pid 2>/dev/null | grep -q "zentek"; then
        idx=$(echo "$map" | grep "$bus" | cut -d, -f1 | tr -d " ")
        name=$(echo $name | tr -d " ")
        echo "  cuda:$idx  PID=$pid  $name  $mem"
    fi
done
'
```

Output example:

```
=== My CUDA GPUs (user: phh) ===

  cuda:0  PID=12345  python  6900 MiB
  cuda:2  PID=12346  python  2100 MiB
```

7. `--models` — List Saved Models

What: List all saved model checkpoint files.

Execution:

```
# RTX Z1
ssh phh@192.168.120.155 'ls -lh ~/magiclab_rl_lab/logs/rsl_rl/magiclab_z1_12dof_velocity/*/model_*.pt'

# Spark G1
ssh spark 'ls -lh ~/IsaacLab/scripts/logs/rsl_rl/g1_pick_place_phase1/*/model_*.pt'
```

8. `--check` — Training Health Check

What: Analyze training metrics trend and present a health report.

Pipeline:

1. Fetch log data (200+ lines) from active training
2. Parse iteration snapshots (blocks separated by `---` lines)
3. Compare earliest vs latest snapshot for key metrics
4. Report trend analysis with actionable advice

Trend signals:

Signal	Condition	Action
Improving	errors trending down, reward up	Continue training
Converging	entropy dropping, errors flat (<1% change)	Nearing convergence
Plateaued	metrics unchanged 5+ snapshots	Converged, consider stopping
Overfitting	errors rising vs earlier snapshots	Risk, stop and rollback

Output example:

```
=== Training Health Check ===

Trend (early -> latest):
  Mean reward:        XX -> YY  (up/down/flat)
  key_error_metric:   XX -> YY  (up/down/flat)

Status: GREEN / YELLOW / ORANGE / RED
Advice: <actionable recommendation>
```

9. `--monitor` — Overfitting Detection & Best Model Finder

What: Run `train_monitor.py` on RTX6000 to detect overfitting and identify best checkpoints. **RTX only.**

Prerequisites (one-time):

```
scp "D:/Desktop_Files/GPU-Train/RTX6000/Magicbot_Z1/magiclab_rl_lab/scripts/train_monitor.py" \
    phh@192.168.120.155:~/magiclab_rl_lab/scripts/train_monitor.py
```

Sub-commands

Sub-command	What it does
(default)	Scan all runs, report health + best model for each

Sub-command	What it does
`--realtime`	Live monitoring of active training run (30s poll)
`--run <DIR>`	Analyze a single specific run
`--start`	Launch continuous background monitor (polls every 120s)
`--stop`	Stop the background monitor process
`--status`	Check if background monitor is running
`--anal`	Deep failure analysis with tuning recommendations

(default) One-shot Analysis

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
```

Auto-downloads `best_models.json` to `D:\Desktop_Files\GPU-Train\RTX6000\Magicbot_Z1\best_models.json` .

Output example:

```
=== Z1 Training Monitor: All Runs ===

  Run                          Iter   Reward  Peak    Best    Status
  ──────────────────────────── ────── ─────── ─────── ─────── ──────────
  v4_gentle_terrain            30100   46.91   46.91  m28500  HEALTHY
  v5_rough_terrain             25400   35.20   35.20  m25400  HEALTHY
  v1_flat                      36400   -0.96   48.00  m28500  OVERFITTING
  ...
```

`--monitor --realtime` — Live Monitoring

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
```

Output format:

```
[16:03:31] iter   3174 | reward   21.68 → | peak 24.90@2246 | best 24.09@2217 | ar -0.656 | ent 16.1 |
```

Auto-exits after 3 consecutive polls with no new data (~90s stale).

`--monitor --start` — Background Monitor

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
```

With auto-export (JIT on overfitting):

```
... --poll_interval 120 --auto_export > /tmp/z1_monitor.log 2>&1 & ...
```

`--monitor --stop`

```
ssh phh@192.168.120.155 "ps aux | grep train_monitor.py | grep -v grep | awk '{print \$2}' | xargs kill
```

`--monitor --status`

```
ssh phh@192.168.120.155 'ps aux | grep train_monitor.py | grep -v grep && echo "---" && tail -5 /tmp/z1
```

`--monitor --anal` — Training Failure Analysis

Prerequisites (one-time):

```
scp "D:/Desktop_Files/GPU-Train/RTX6000/Magicbot_Z1/magiclab_rl_lab/scripts/train_analyzer.py" \
    phh@192.168.120.155:~/magiclab_rl_lab/scripts/train_analyzer.py
```

Analyze all failed runs:

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
```

Analyze specific run:

```
ssh phh@192.168.120.155 "... python -u scripts/train_analyzer.py --run_dir logs/rsl_rl/magiclab_z1_12dc
```

6 Failure Modes:

Mode	Name	Severity	Key Signals
`REWARD_DECLINE`	奖励渐退	MODERATE	reward_slope < 0, time_out > 50%
`POLICY_COLLAPSE`	策略崩溃	CRITICAL	bad_ori > 80%, ep_len < 100, reward < 0
`ACTION_EXPLOSION`	动作爆炸	CRITICAL	action_rate < -5.0, ep_len 急降
`ENTROPY_COLLAPSE`	熵坍缩	HIGH	entropy 下降 > 90%, absolute < 0.05
`VALUE_DIVERGE`	价值函数发散	HIGH	value_loss > 100 且持续上升
`HIGH_FALL_RATE`	高频摔倒	CRITICAL	bad_ori > 80%, reward >= 0, time_out < 50%

Detection Rules

5 independent signals (any one triggers alert):

Signal	Threshold	Meaning
Reward decline	>20% from peak	Policy degraded
action_rate	< -1.0 (gentle)	Jitter too high
Policy std	< 0.01	Actions deterministic/extreme
Value loss	> 100	Value function diverged
Entropy collapse	>80% decline from peak	Policy too certain

Thresholds auto-adjust by terrain: `--terrain flat|gentle|rough` .

10. `--sim` — Record Simulation Video

What: Record both Isaac Lab and MuJoCo simulation videos from a checkpoint. All recording on RTX 6000D.

Auto-resolve Mode (`--best <VERSION>`)

```
/gpu-train --sim --best s2_gentle              # Auto-resolve from best_models.json (HEALTHY)
/gpu-train --sim --best s4_full_terrain --video_length 400  # Best reward overall
/gpu-train --sim --best s2_gentle --mujoco_only
/gpu-train --sim --best s2_gentle --isaac_only
```

Resolves checkpoint from `best_models.json` :

```
ssh phh@192.168.120.155 'cat ~/magiclab_rl_lab/logs/rsl_rl/magiclab_z1_12dof_velocity/best_models.json'
```

Available versions (from best_models.json):

Version	Best Model	Reward	Peak	Status
`s4_full_terrain`	model_8100	67.64	67.64@8164	OVERFITTING
`s2_gentle`	model_47862	47.06	48.1@47857	HEALTHY
`s1_flat`	model_3861	47.33	48.35@2713	OVERFITTING
`s3_rough_l2`	model_32790	38.04	38.94@32790	OVERFITTING

Manual Mode (`--checkpoint <PATH>`)

When `--best` is NOT specified, requires explicit `--checkpoint` path.

Parameters:

Parameter	Default	Description
`--checkpoint`	—	Model checkpoint path
`--best <VERSION>`	—	Auto-resolve from best_models.json
`--vel_x`	0.5	Forward velocity (m/s)
`--vel_y`	0.0	Lateral velocity (m/s)
`--vel_yaw`	0.0	Yaw angular velocity (rad/s)
`--duration`	10	MuJoCo simulation duration (s)
`--video_length`	200	Isaac Sim video steps
`--mujoco_only`	—	MuJoCo recording only
`--isaac_only`	—	Isaac Sim recording only
`--skip_export`	—	Skip JIT export

Pipeline Steps

Step 1: Export JIT on RTX (skip with `--skip_export`)

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
```

Step 2: Isaac Sim Recording (headless)

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
    --checkpoint <CHECKPOINT_PATH> --video --video_length <VIDEO_LENGTH> --headless --num_envs 1 --devi
```

Step 3: MuJoCo Recording (EGL offscreen)

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
    --mjcf ~/magicbot-z1_description/mjcf/MAGICBOTZ1.xml \
    --checkpoint <CHECKPOINT_PATH> \
    --record /tmp/<SAVE_NAME>_mujoco.mp4 \
    --duration <DURATION> --vel_x <VEL_X> --headless"
```

Step 4: Download Videos

```
SAVE_DIR="D:/Desktop_Files/GPU-Train/RTX6000/Magicbot_Z1/videos/<VERSION_NAME>"
mkdir -p "$SAVE_DIR"

scp phh@192.168.120.155:<checkpoint_dir>/videos/play/rl-video-step-0.mp4 \
    "$SAVE_DIR/<SAVE_NAME>_rtx_isaaclab.mp4"
scp phh@192.168.120.155:/tmp/<SAVE_NAME>_mujoco.mp4 "$SAVE_DIR/"
```

Step 5: Add Labels (mandatory)

```
python "D:/Desktop_Files/GPU-Train/RTX6000/Magicbot_Z1/scripts/label_video.py" "$VIDEO_FILE" \
    --model <MODEL_NAME> --run <RUN_NAME> --reward <REWARD> \
    --terrain <TERRAIN> --iteration <ITER_NUM> --action-mean <ACTION_MEAN_ABS>
```

Quick Options

Flag	Steps Executed
(default)	1→2→3→4→5
`--best <VERSION>`	1→2→3→4→5
`--mujoco_only`	1→3→4→5
`--isaac_only`	1→2→4→5
`--skip_export`	2→3→4→5

One-click Script

```
bash D:/Desktop_Files/GPU-Train/RTX6000/rtx_record_video.sh <RUN_DIR> <CHECKPOINT> [VIDEO_LENGTH] [VEL_
```

Prerequisites (one-time)

Server	Requirements
RTX	`pip install mujoco imageio imageio-ffmpeg` ; `~/magicbot-z1_description/mjcf/MAGICBOTZ1.xml`
Local	`av` , `imageio` , `imageio-ffmpeg` , `Pillow` ; `rtx_record_video.sh` ; `label_video.py`

Common Issues

Symptom	Fix
`size mismatch` in export	Check checkpoint format (rsl-rl 3.x vs 5.x)
Robot falls in MuJoCo	Use `mujoco_sim2sim.py` (has sim2sim fixes)
Isaac Sim video all black	Renderer warmup — use updated `play_z1_video.py`
Isaac Sim robot not visible	Camera stuck — use updated `play_z1_video.py`
RTX Isaac Sim `--video` fails	Do NOT use xvfb-run; just `--headless` is enough
MuJoCo EGL fails	Check `~/miniconda3/envs/isaaclab/share/glvnd/egl_vendor.d/10_nvidia.json`

11. `--start` — Launch / Resume Training

What: Start a new training run or resume an interrupted one. Auto-detects platform.

Sub-commands

Sub-command	Description
`--resume`	Find latest run's latest checkpoint and continue
`--from <VERSION>`	Start new run from a known version's best model
(no sub-command)	Interactive: ask for checkpoint source and run name

GPU Selection (Interactive)

When `--gpus <N>` is specified → use N GPUs directly.
When omitted → interactive selection:

1. Run `--idle` to check free GPUs
2. Find **max contiguous idle block from GPU 0** (torchrun limitation)
3. Ask user how many GPUs to use

Finding contiguous block:

```
Idle GPUs: [0, 1, 3, 4, 6, 7]
Scan from 0: 0✓ 1✓ 2✗ → max contiguous = 2 (cuda:0-1)
```

Output example:

```
=== GPU Idle Check (RTX Server) ===

  GPU  Status     Util%  Mem Used / Total
  0    IDLE       0%     0.1G / 85.7G
  1    IDLE       0%     0.1G / 85.7G
  2    IDLE       0%     0.1G / 85.7G
  3    IDLE       0%     0.1G / 85.7G
  4    busy       95%    64.3G / 85.7G
  ...

  Max contiguous block: 4 GPUs (cuda:0-3)

How many GPUs to use for training?
  [1]  1 GPU   [2]  2 GPUs   [3]  3 GPUs   [4]  4 GPUs  ← Recommended
```

num_envs Selection (RTX only)

After GPU count determined, auto-estimate optimal `--num_envs`:

1. Query GPU memory
2. Calculate available per GPU
3. Estimate: ~610 envs/GB for Z1 12DOF
4. Max safe: `floor(avail_GB * 610 * 0.70)`

Spark: Always `--num_envs 64` (4096 causes OOM on GB10).

Options offered:

```
  [1]  4096   — ~6.7G/GPU  (current default)
  [2]  8192   — ~13G/GPU
  [3]  16384  — ~25G/GPU    ← Recommended
  [4]  32768  — ~50G/GPU
```

Multi-GPU: `--gpus <N>`

- Uses `torchrun --nproc_per_node=N`
- GPUs assigned sequentially from GPU 0 (cannot skip)
- **Do NOT use `CUDA_VISIBLE_DEVICES`** — causes Isaac Sim hang

`--start --resume` — Resume Latest Run

Single-GPU:

```
LATEST_RUN=$(ssh phh@192.168.120.155 'ls -td ~/magiclab_rl_lab/logs/rsl_rl/magiclab_z1_12dof_velocity/2
RUN_DIR=$(basename "$LATEST_RUN")
LATEST_MODEL=$(ssh phh@192.168.120.155 "ls -t ${LATEST_RUN}/model_*.pt 2>/dev/null | head -1")
MODEL_FILE=$(basename "$LATEST_MODEL")

ssh phh@192.168.120.155 "cd ~/magiclab_rl_lab && source ~/miniconda3/etc/profile.d/conda.sh && conda ac
```

Multi-GPU:

```
ssh phh@192.168.120.155 "cd ~/magiclab_rl_lab && source ~/miniconda3/etc/profile.d/conda.sh && conda ac
```

`--start --from <VERSION>` — Start from Known Checkpoint

1. Read `best_models.json` from server
2. Find matching version (partial match)
3. Confirm with user
4. Launch with new run name

EGL Pre-flight Check

```
ssh phh@192.168.120.155 'test -f ~/miniconda3/envs/isaaclab/share/glvnd/egl_vendor.d/10_nvidia.json &&
```

If MISSING:

```
ssh phh@192.168.120.155 'cp /usr/share/glvnd/egl_vendor.d/10_nvidia.json ~/miniconda3/envs/isaaclab/sha
```

Default Parameters

RTX Z1 Locomotion:

Parameter	Single-GPU	Multi-GPU
Script	`train.py`	`train_multigpu.py`
Launcher	`python -u`	`torchrun --nproc_per_node=N`
`--distributed`	No	Yes
`--device`	`cuda:0`	Auto

Parameter	Single-GPU	Multi-GPU
`--num_envs`	User-selected	User-selected per GPU
`--max_iterations`	50000	50000
`--headless`	Always	Always
Log path	`/tmp/z1_train_<name>.log`	`/tmp/z1_mgpu_<name>.log`

Spark G1 PickPlace:

Parameter	Value
Script	`train_g1_pickplace.py`
`--num_envs`	**64** (hardcoded)
`--headless`	Always
tmux session	`g1_train`

Launch Output Example

```
=== Training Launched ===

  Run:       v6_l1_action_rate
  Checkpoint: model_1700.pt (from v6_l1_action_rate)
  PID:       3438217
  GPUs:      4 (cuda:0, cuda:1, cuda:2, cuda:3)
  num_envs:  16384 per GPU (65,536 total)
  Log:       /tmp/z1_mgpu_v6_4gpu.log

  EGL:       ✓ (NVIDIA vendor file present)
  Speed:     ~400,000 steps/s (4 GPUs × 16384 envs)
  ETA:       ~1-2 hours
```

12. `--play` — Live Play on Spark Display

What: Launch interactive policy evaluation on Spark's physical display with camera tracking. **Spark only.**

Usage

```
/gpu-train --play --best v4_gentle                    # Play best model
/gpu-train --play --best v4_gentle --num_envs 4       # 4 robots
/gpu-train --play --best v4_gentle --camera_distance 2.0
/gpu-train --play --best v4_gentle --max_steps 2000
```

Parameters

Parameter	Default	Description
`--best <VERSION>`	Required	Auto-resolve checkpoint
`--num_envs <N>`	1	Number of parallel robots
`--max_steps <N>`	1000	Simulation steps
`--camera_distance <M>`	3.5	Camera distance
`--camera_height <M>`	1.5	Camera height
`--no_camera_track`	Off	Disable camera tracking
`--real_time`	Off	Run at real-time speed

Pipeline

Step 1: Check exported policy on Spark

```
ssh spark 'ls ~/magiclab_rl_lab/logs/rsl_rl/magiclab_z1_12dof_velocity/<RUN_DIR>/exported/policy.pt 2>/
```

Step 2: Launch play

```
ssh spark "export DISPLAY=:1 && export LD_PRELOAD=/lib/aarch64-linux-gnu/libgomp.so.1 && source ~/minic
    --task Magiclab-Z1-12dof-Velocity \
    --policy logs/rsl_rl/magiclab_z1_12dof_velocity/<RUN_DIR>/exported/policy.pt \
    --num_envs <NUM_ENVS> --max_steps <MAX_STEPS> \
    --camera_distance <DIST> --camera_height <HEIGHT> \
    --device=cuda:0 > /tmp/z1_play_<VERSION>.log 2>&1 & echo PID=\$!"
```

GPU Memory Scaling (Spark GB10)

num_envs	Est. VRAM	Status
1	~8 GB	Safe

num_envs	Est. VRAM	Status
4	~15 GB	Safe
8	~25 GB	Safe
16	~45 GB	OK
32	~80 GB	Tight

Output Example

```
=== Spark Live Play ===

  Model:      v4_gentle_terrain → model_47862.pt
  Reward:     47.06 (HEALTHY)
  PID:        476740
  Display:    :1 (Spark screen)

  Robots:     4 (num_envs=4)
  Steps:      1000
  Camera:     tracking ON (dist=3.5, height=1.5)
  GPU:        ~15 GB / 92 GB
```

13. `--kill` — Stop Training

What: Safely stop a running training process. Confirms with user before killing.

RTX:

```
ssh phh@192.168.120.155 'ps aux | grep "train.py" | grep phh | grep -v grep'
# Confirm, then:
ssh phh@192.168.120.155 'kill <PID>'
```

Spark:

```
ssh spark 'tmux kill-session -t g1_train 2>/dev/null && echo "Stopped" || echo "Not found"'
```

14. `--automation` — 5-Phase Automated Training Pipeline

What: Manage the multi-phase automated training pipeline on RTX. Wraps `phase_orchestrator.py` for full lifecycle. **RTX only.**

Sub-commands

Sub-command	Description
`--start`	Launch the 5-phase pipeline (fresh)
`--status`	Check pipeline status + current sub-phase progress
`--tail`	Show pipeline orchestrator log tail
`--stop`	Stop pipeline (kill orchestrator + training)
`--resume`	Resume pipeline from saved state
`--dry-run`	Dry run (print all 10 sub-phase configs)

Parameters

Parameter	Default	Description
`--plan <yaml>`	`training_plans/z1_5phase_plan.yaml`	Plan file path
`--gpus <N>`	4	Number of GPUs
`--from <SUB_PHASE>`	p1_coarse	Start from specific sub-phase
`--fresh`	Off	Ignore saved state
`--poll <N>`	120	Monitor poll interval (s)

`--automation --start`

Launch:

```
PLAN="training_plans/z1_5phase_plan.yaml"
NUM_GPUS=4
POLL=120

ssh phh@192.168.120.155 "cd ~/magiclab_rl_lab && source ~/miniconda3/etc/profile.d/conda.sh && conda ac
```

With `--from` :

```
... --start-from ${SUB_PHASE} --fresh --poll-interval ${POLL} ...
```

Output:

```
=== 5-Phase Pipeline Launched ===

  Plan:      training_plans/z1_5phase_plan.yaml
  Start:     p1_coarse (fresh)
  PID:       3438217
  GPUs:      4 (cuda:0-3)
  Poll:      120s
  Log:       /tmp/z1_5phase_pipeline.log

  Phases:    p1 → p2 → p3 → p4 → p5 (10 sub-phases, ~210K iter)
```

--automation --status

```
ssh phh@192.168.120.155 'ps aux | grep phase_orchestrator | grep -v grep'
ssh phh@192.168.120.155 'cat ~/magiclab_rl_lab/orchestrator_state.json 2>/dev/null'
ssh phh@192.168.120.155 'tail -15 /tmp/z1_5phase_pipeline.log 2>/dev/null'
ssh phh@192.168.120.155 'tail -30 ~/magiclab_rl_lab/logs/train_*.log 2>/dev/null | tail -30'
```

Output:

```
=== 5-Phase Pipeline Status ===

  Orchestrator:  PID 3438217  ● Running
  Current:       p1_coarse (P1 Coarse — Bootstrap)
  Run dir:       2026-05-06_15-47-12_p1_coarse
  GPUs:          4 × RTX 6000

  Progress:
    [p1_coarse] iter 725/5000  reward 14.76  peak 15.08  HEALTHY  ← current
    [p1_fine  ] pending
    ...
    [p5_fine  ] pending

  Best model:    model_718.pt (reward: 15.08)
  Rollbacks:     0
```

`--automation --tail`

```
ssh phh@192.168.120.155 'tail -40 /tmp/z1_5phase_pipeline.log 2>/dev/null'
```

Also shows last 30 lines of active training log.

`--automation --stop`

1. Find orchestrator + training PIDs
2. Confirm with user
3. Kill processes (orchestrator first)
4. State file preserved for resume

```
ssh phh@192.168.120.155 'kill <ORCH_PID> <TRAIN_PID>'
```

`--automation --resume`

```
ssh phh@192.168.120.155 "cd ~/magiclab_rl_lab && source ~/miniconda3/etc/profile.d/conda.sh && conda ac
```

Do NOT pass `--fresh` — reads `orchestrator_state.json` to determine resume point.

`--automation --dry-run`

```
ssh phh@192.168.120.155 "source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab && cd ~/
```

Pipeline Architecture

```
phase_orchestrator.py
  ├── phase_manager.py       — Parse YAML, 3-layer merge (defaults → phase → sub_phase)
  ├── config_generator.py    — Generate velocity_env_cfg.py from params dict
  ├── ppo_override.py        — Generate temp PPO config (LR, entropy override)
  ├── training_launcher.py   — torchrun launch with --agent_cfg
  ├── embedded_monitor.py    — TensorBoard polling, overfitting detection
  └── state_store.py         — JSON state persistence for resume

Files on RTX:
  Pipeline log:    /tmp/z1_5phase_pipeline.log
  State file:      ~/magiclab_rl_lab/orchestrator_state.json
  Training log:    ~/magiclab_rl_lab/logs/train_<sub_phase>.log
  Run dirs:        ~/magiclab_rl_lab/logs/rsl_rl/magiclab_z1_12dof_velocity/<run_dir>/
  Generated cfgs:  ~/magiclab_rl_lab/tmp/phase_configs/<sub_phase>/
```

5-Phase Curriculum

```
p1 (flat, bootstrap)          From scratch
  p1_coarse → p1_fine → video
    ↓ resume from best
p2 (flat, velocity tracking)
  p2_coarse → p2_fine → video
    ↓
p3 (gentle terrain)
  p3_coarse → p3_fine → video
    ↓
p4 (rough terrain)
  p4_coarse → p4_fine → video
    ↓
p5 (full terrain + polish)
  p5_coarse → p5_fine → video → final model
```

10 sub-phases, ~210K total iterations.

Auto-Rollback

```
At sub-phase completion:
  if best_reward < starting_reward × 0.95:
      → discard results, retry from starting checkpoint (LR × 0.5)
      → max 1 retry
  else:
      → advance to next sub-phase using best checkpoint
```

15. `--connect` — Test Server Connectivity

What: Test SSH to both servers. Auto-launch VPN if connection fails.

Execution:

```bash
# Parallel test
ssh -o ConnectTimeout=10 spark 'echo "Spark OK - $(hostname)"'
ssh -o ConnectTimeout=10 phh@192.168.120.155 'echo "RTX OK - $(hostname)"'
```

On failure:

- Spark → launch aTrust (`C:\Users\Public\Desktop\aTrust.lnk`), retry every 5s × 6

- RTX → launch iNode (`C:\Program Files (x86)\iNode\iNode Client\iNode Client.exe`), retry every 3s × 10

Output:

```
=== Server Connectivity ===

  Server    SSH                      Status
  ──────    ─────────────────────    ──────────────────
  Spark     spark (59.66.25.192)     OK  (spark-10d3)
  RTX       phh@192.168.120.155      OK  (pro6000d)

  VPN: aTrust ✓  iNode ✓
```

16. `--update` — Update Training Log

What: Update `training_log_YYYY-MM-DD.md` in working directory.

Pipeline:

1. Check if log file exists; create with header if not
2. Gather data from remote: process status, log tail, model list
3. Append a new numbered section with:
   - Process status, elapsed time, ETA
   - Key training metrics table
   - Saved models list
4. Only record successful data (no errors/crashes)

17. `--compare` — Video Comparison Grid (Deprecated)

> **Deprecated**: Use `/merge` skill instead.

```
/merge videos/s2_flat/*.mp4 videos/s3_gentle/*.mp4 -t 2x2 --labels "v1" "v2" "v3" "v4"
```

Server Environment Details

RTX Server

Property	Value
SSH	`phh@192.168.120.155` (VPN required)
Architecture	x86_64
GPU	8x NVIDIA RTX 6000D (85 GB each)
Conda Env	`isaaclab`
Conda Activation	`source ~/miniconda3/etc/profile.d/conda.sh && conda activate isaaclab`
Project Root	`~/magiclab_rl_lab`
Isaac Lab Root	`~/IsaacLab`
Display	Headless offscreen (`--headless` , no Xvfb)
Log Path	`/tmp/z1_train_*.log`
Checkpoint Path	`~/magiclab_rl_lab/logs/rsl_rl/magiclab_z1_12dof_velocity/<run_dir>/`

Key Config Files (on server):

File	Path
Robot config	`source/magiclab_rl_lab/magiclab_rl_lab/assets/robots/magiclab.py`
Env config	`source/magiclab_rl_lab/magiclab_rl_lab/tasks/locomotion/robots/z1/12dof/velocity_env_cfg.py`
Agent config	`source/magiclab_rl_lab/magiclab_rl_lab/tasks/locomotion/agents/rsl_rl_ppo_cfg.py`
Train script	`scripts/rsl_rl/train.py`

Spark Server

Property	Value
SSH	`spark` (alias in `~/.ssh/config`) → `59.66.25.192`

Property	Value
User	`zentek`
Architecture	ARM (aarch64)
GPU	NVIDIA GB10 (~92 GB)
Conda Env	`env_isaaclab`
Conda Activation	`source ~/miniconda3/etc/profile.d/conda.sh && conda activate env_isaaclab`
Isaac Lab Root	`~/IsaacLab`
Display	`DISPLAY=:1` (GNOME desktop)
LD_PRELOAD	`/lib/aarch64-linux-gnu/libgomp.so.1`
num_envs	**Always 64** for G1 PickPlace (4096 causes OOM)
VPN	aTrust VPN

Checkpoint Paths:

Project	Path
G1 PickPlace	`~/IsaacLab/scripts/logs/rsl_rl/g1_pick_place_phase1/<run_dir>/`
K1 Fight	`~/booster_train/logs/rsl_rl/k1_fight_001/<run_dir>/`

Common Issues:

Server	Symptom	Fix
Spark	Silent crash with `--video`	Remove `--headless`, use `DISPLAY=:1`
Spark	GLFW init failed	Set `DISPLAY=:1`
RTX	SSH timeout	Connect iNode VPN
RTX	`KeyError: 'actor'`	rsl-rl version mismatch → `rsl-rl-lib==3.0.1`
RTX	iray permission error	`rm -rf ~/.local/share/ov/data/exts/v2/omni.iray.libs-*`